Claudia Alin Rojas Jiménez

Bitácora S08: Saga Orquestada y Circuit Breaker

1. El problema

Antes de implementar la saga, simulamos una caída del servicio de cuenta justo en medio de una transferencia usando una pausa de 20 segundos.

Como cada servicio tiene su propia base de datos, el @Transactional ya no abarca ambas operaciones. El cargo se confirmó de forma independiente, pero el abono falló por la caída del servicio,

```
$ curl -sS -X POST localhost:8080/api/cuentas -H 'Content-Type: application/json' -d '{"clabe":"1111111111111111","titular":"Origen",
"saldoInicial":1000}'
{"id":14,"clabe":"1111111111111111","titular":"Origen","saldo":1000}
```

```
$ curl -sS -X POST localhost:8080/api/cuentas -H 'Content-Type: application/json' -d '{"clabe":"2222222222222222","titular":"Destino"
,"saldoInicial":500}'
{"id":15,"clabe":"2222222222222222","titular":"Destino","saldo":500}
```

En estas capturas podemos observar que la cuenta 1 tiene 1000 pesos en su saldo y la cuenta 2 tiene 500

```
$ curl -i -X POST localhost:8080/api/transferencias -H 'Content-Type: application/json' -d '{"claveOrigen":"1111111111111111","claveD
estino":"2222222222222222","monto":200}'
HTTP/1.1 504 Gateway Timeout
Content-Type: application/json
Content-Length: 139

{"timestamp":"2026-10-11T06:14:22.312+00:00","path":"/api/transferencias","status":504,"error":"Gateway Timeout","requestId":"28176e92-
76"}
```

Al realizar una transferencia de 1 a 2 con el servicio de cuentas apagado, obtenemos el error 504.

```
$ curl localhost:8080/api/cuentas/1111111111111111
{"id":14,"clabe":"1111111111111111","titular":"Origen","saldo":800.00}
```

```
$ curl localhost:8080/api/cuentas/2222222222222222
{"id":15,"clabe":"2222222222222222","titular":"Destino","saldo":500.00}
```

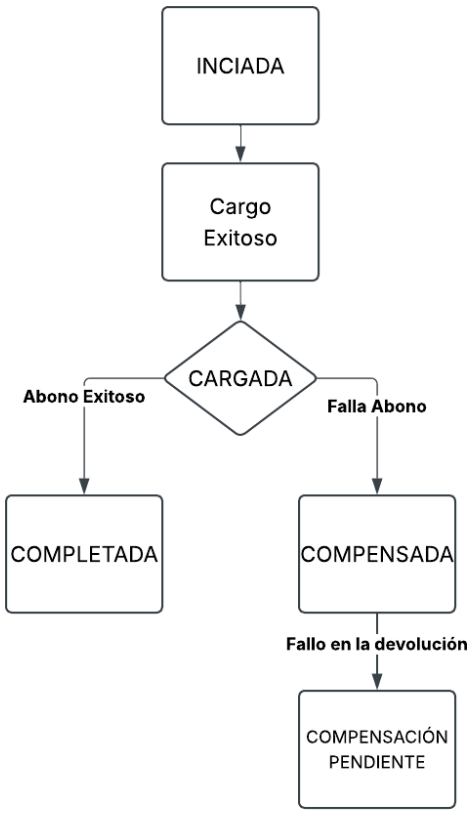
Al volver a iniciar el servicio y consultar los saldos, podemos observar cómo el cobro a la cuenta 1 se realizó pero no llegó a 2 y no regresó a la cuenta de origen.

Claudia Alin Rojas Jiménez

2. La Saga Orquestada

Para solucionar el problema, convertimos a TransferenciaService en el orquestador de la operación, dándole la responsabilidad de rastrear el avance y ejecutar las compensaciones necesarias.

Diagrama de estados de la transferencia:



3. El Código Clave

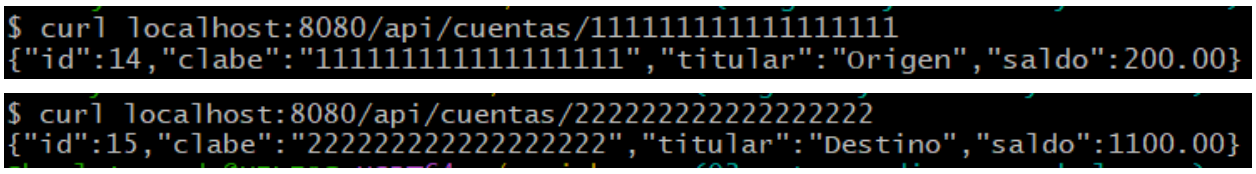
Añadimos la columna estado en la entidad Transferencia.java y modificamos TransferenciaService.java para envolver las operaciones en bloques try-catch:

```java
// Fragmento clave de TransferenciaService.java
try {
    // 2. Ejecutar el cargo al origen
    CuentaRemota origen = cuentas.cargar(claveOrigen, monto);
    guardada.setEstado("CARGADA");
    transferencias.save(guardada);
```

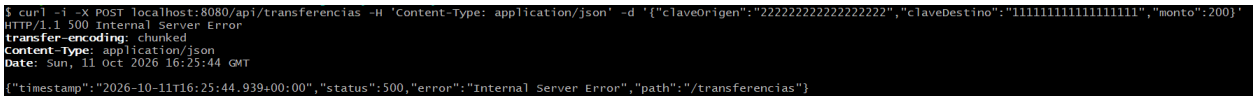
```
    // 4. Ejecutar el abono al destino
    CuentaRemota destino = cuentas.abonar(claveDestino, monto);
    guardada.setEstado("COMPLETADA");
    transferencias.save(guardada);
    // ... notificaciones
} catch (Exception e) {
    // COMPENSACIÓN: El abono falló, devolvemos el dinero
    try {
        cuentas.abonar(claveOrigen, monto);
        guardada.setEstado("COMPENSADA");
        transferencias.save(guardada);
    } catch (Exception ex) {
        // FALLA LA COMPENSACIÓN
        guardada.setEstado("COMPENSACION_PENDIENTE");
        transferencias.save(guardada);
    }
    throw new RuntimeException("Fallo: " + guardada.getEstado(), e);
}
```

4. La compensación funcionando

Forzamos un fallo en el abono inyectando la variable SIMULAR_FALLO_ABONO=true. Al fallar el abono, el servicio atrapó el error y ejecutó la transacción de reversa (abonar el dinero de vuelta a la cuenta origen).



En un inicio, la cuenta 1 tiene un saldo de 200 y la cuenta 2 tiene 1100



Si realizamos una transferencia de 2 a 1 simulando el fallo obtenemos el error 500

```
$ docker compose exec db psql -U mexibanco -d transferencia -c "SELECT * FROM transferencia;"
 id |   clave_destino    |    clave_origen    |            fecha            | monto  |   estado
----+--------------------+--------------------+-----------------------------+--------+------------
  1 | 002180009117992002 | 002180009117992001 | 2026-10-05 05:58:43.657436+00 | 200.00 |
  2 | 002180009118181002 | 002180009118181001 | 2026-10-05 06:30:12.571047+00 | 200.00 |
  3 | 222222222222222222 | 111111111111111111 | 2026-10-11 06:34:07.160208+00 | 200.00 | COMPLETADA
 36 | 222222222222222222 | 111111111111111111 | 2026-10-11 15:56:04.170385+00 | 200.00 | COMPLETADA
 37 | 222222222222222222 | 111111111111111111 | 2026-10-11 15:58:01.466062+00 | 200.00 | COMPLETADA
 38 | 111111111111111111 | 222222222222222222 | 2026-10-11 16:21:48.934279+00 | 200.00 | COMPENSADA
 39 | 111111111111111111 | 222222222222222222 | 2026-10-11 16:25:44.899063+00 | 200.00 | COMPENSADA
(7 rows)
```

Si consultamos las transferencias en la DB, podemos observar que las transferencias realizadas con la simulación del fallo aparecen con el estado COMPENSADA satisfactoriamente.

5. Cuando la compensación también falla

Apagamos el contenedor cuenta justo durante la pausa de 20 segundos para simular que el sistema se quedó sin conexión en el instante exacto en que intentaba devolver el dinero al origen. El bloque catch anidado lo atrapó y marcó el estado correcto.

```
$ PAUSA_ENTRE_CARGO_Y_ABONO_MS=20000 docker compose up -d transferencia
[+] up 3/3
 ✔ Container mexi-banco-v06-discovery-1     Healthy                    3.9s
 ✔ Container mexi-banco-v06-db-1            Healthy                    3.9s
 ✔ Container mexi-banco-v06-transferencia-1 Started                    4.1s
```

```
$ curl -i -X POST localhost:8080/api/transferencias -H 'Content-Type: application/json' -d '{"claveOrigen":"222222222222222222
","claveDestino":"111111111111111111","monto":200}'
HTTP/1.1 504 Gateway Timeout
Content-Type: application/json
Content-Length: 140

{"timestamp":"2026-10-11T16:28:11.729+00:00","path":"/api/transferencias","status":504,"error":"Gateway Timeout","requestId":"
6b1b033b-470"}
```

Si hacemos una transferencia de 2 a 1 mientras el servicio de cuentas está apagado obtenemos el error 504.

```
$ docker compose exec db psql -U mexibanco -d transferencia -c "SELECT * FROM transferencia;"
 id |   clave_destino    |    clave_origen    |            fecha            | monto  |         estado
----+--------------------+--------------------+-----------------------------+--------+------------------------
  1 | 002180009117992002 | 002180009117992001 | 2026-10-05 05:58:43.657436+00 | 200.00 |
  2 | 002180009118181002 | 002180009118181001 | 2026-10-05 06:30:12.571047+00 | 200.00 |
  3 | 222222222222222222 | 111111111111111111 | 2026-10-11 06:34:07.160208+00 | 200.00 | COMPLETADA
 36 | 222222222222222222 | 111111111111111111 | 2026-10-11 15:56:04.170385+00 | 200.00 | COMPLETADA
 37 | 222222222222222222 | 111111111111111111 | 2026-10-11 15:58:01.466062+00 | 200.00 | COMPLETADA
 38 | 111111111111111111 | 222222222222222222 | 2026-10-11 16:21:48.934279+00 | 200.00 | COMPENSADA
 39 | 111111111111111111 | 222222222222222222 | 2026-10-11 16:25:44.899063+00 | 200.00 | COMPENSADA
 40 | 111111111111111111 | 222222222222222222 | 2026-10-11 16:27:56.890982+00 | 200.00 | COMPENSACION_PENDIENTE
(8 rows)
```

Al consultar tenemos correctamente el estado de COMPENSACION_PENDIENTE

Para resolver la compensación pendiente, implementaría un proceso en segundo plano asíncrono como una tarea programada que lea la base de datos periódicamente y reintente el abono de compensación hasta que el servicio de cuenta vuelva a estar disponible.

La compensación debe ser estrictamente idempotente porque los errores de red son ambiguos, si el microservicio de cuenta sí recibió y aplicó la devolución pero la respuesta de éxito se

perdió en la red, se intentaría devolver el dinero una segunda vez. Si no es idempotente, el banco terminaría regalando dinero por cada reintento.

6. El Circuit Breaker

Para evitar agotar los recursos de transferencia esperando timeouts cuando cuenta está caído, implementamos Resilience4j.

- Estado CERRADO (CLOSED): El servicio de cuenta está sano y las peticiones fluyen normalmente.

```
$ docker compose exec transferencia curl -s localhost:8080/actuator/circuitbreakers
{"circuitBreakers":{"cuenta":{"failureRate":"-1.0%","slowCallRate":"-1.0%","failureRateThreshold":"50.0%","slowCallRateThresho
ld":"100.0%","bufferedCalls":0,"failedCalls":0,"slowCalls":0,"slowFailedCalls":0,"notPermittedCalls":0,"state":"CLOSED"}}}
```

- Estado ABIERTO (OPEN): El servicio de cuenta se apagó. Al superar el umbral del 50% de fallas en la ventana de 5 peticiones, el circuito se abrió. Las peticiones comenzaron a ser rechazadas instantáneamente (fail-fast) protegiendo al orquestador.

```
$ docker compose exec transferencia curl -s localhost:8080/act
uator/circuitbreakers
{"circuitBreakers":{"cuenta":{"failureRate":"100.0%","slowCallRate":"0.0%","failureRateThreshold":"50.0%","slowCallRateThresho
ld":"100.0%","bufferedCalls":5,"failedCalls":5,"slowCalls":0,"slowFailedCalls":0,"notPermittedCalls":1,"state":"OPEN"}}}
```

- Estado SEMIABIERTO (HALF_OPEN): Tras esperar el wait-duration de 15 segundos con la cuenta apagada, se lanzó una petición. El circuito permitió el paso de esta petición de prueba para validar si el servicio ya se había recuperado.

```
$ docker compose exec transferencia curl -s localhost:8080/actuator/circuitbreakers
{"circuitBreakers":{"cuenta":{"failureRate":"-1.0%","slowCallRate":"-1.0%","failureRateThreshold":"50.0%","slowCallRateThresho
ld":"100.0%","bufferedCalls":1,"failedCalls":1,"slowCalls":0,"slowFailedCalls":0,"notPermittedCalls":0,"state":"HALF_OPEN"}}}
```

7. Lo que la saga no te da

¿Qué ve alguien que consulta el saldo de origen entre el cargo y la compensación?

Si el usuario consulta su saldo en medio del proceso, verá que tiene menos dinero del que realmente le corresponde, aunque la transferencia vaya a ser cancelada y compensada segundos después. La saga nos garantiza la consistencia eventual, pero sacrifica el Aislamiento de las propiedades ACID. El estado intermedio queda expuesto al público.